

Roblox & Software Engineering Series

A 5-post progression from gameplay systems to AI and broader software architecture.

Post 1 — Pet Simulator

Pet Simulator

One of my first larger Roblox projects was a small pet-simulator-style game.

I built the systems connecting the UI, pets, and gameplay logic, which gave me a much better understanding of how different parts of a game communicate with each other.

At the time, I was mainly focused on making the game work.

Looking back at it now, I can see plenty of things I'd structure differently — but that's also what makes old projects useful.

They show how much your approach changes as you gain experience.

This project was one of the things that got me interested in building more complex gameplay systems with Luau.

[#Roblox](#) [#GameDevelopment](#) [#Luau](#) [#Programming](#) [#SoftwareDevelopment](#)

Post 2 — Time Rewind

Time Rewind

I wanted to experiment with a mechanic that sounds simple but quickly becomes complicated:

■ Rewinding time.

For this Roblox project, I worked on a system that allowed gameplay state to be recorded and then reconstructed when the player activated the rewind mechanic.

The interesting part wasn't the rewind button itself.

It was figuring out what actually needs to be saved and how to restore that state without everything falling apart.

Projects like this taught me to think less about individual features and more about the underlying state of a system.

It's also one of the projects I'd approach very differently today.

[#Roblox](#) [#GameDev](#) [#Luau](#) [#GameProgramming](#) [#Programming](#)

Post 3 — Wall Movement

Wall Movement

I spent some time experimenting with custom character movement in Roblox.

One of the mechanics I worked on was wall-running / wall-pinning.

The project involved detecting nearby surfaces with raycasts and using that information to control the character's movement and orientation.

What looked like a simple movement mechanic ended up involving quite a few questions:

- How do I reliably detect the wall?
- How should the character orient itself?
- What happens when the player leaves the wall?
- How should velocity be handled?
- How do I make the movement feel responsive?

This was a good introduction to the fact that game development often involves combining relatively simple systems to produce something that feels complex.

[#Roblox](#) [#GameDevelopment](#) [#Luau](#) [#GamePhysics](#) [#Programming](#)

Post 4 — Worm Boss

Worm Boss

One of my more ambitious Roblox projects was a Terraria-style segmented worm boss.

The boss used a finite-state machine for its behavior, while the body consisted of many independently represented segments.

The networking side became an interesting challenge.

I experimented with server-authoritative movement and client-side replication/interpolation so that the segmented body could move smoothly without simply relying on the client to control everything.

Working on this project taught me a lot about the difference between:

"It works on my screen."

and

"It works as a multiplayer system."

It's an older project now, and there are definitely architectural decisions I'd change today.

But looking back at it, it's probably one of the projects that taught me the most about game architecture.

#Roblox #GameDevelopment #Luau #GameAI #Networking #Programming

Post 5 — The Transition

What Came Next

Looking back at some of my older Roblox projects made me realize something.

I started game development because I wanted to make things move.

Then I became interested in how those systems actually work.

Movement led to physics.

Physics led to simulation.

Gameplay AI led me toward algorithms and decision-making.

Networking made me think more about distributed systems and architecture.

And eventually, that curiosity expanded beyond games.

Now I'm working on projects involving AI, automation, and software architecture — including my own desktop AI assistant.

I still enjoy game development, but I'm increasingly interested in the underlying engineering behind the things I build.

It's interesting to look back at old projects and realize that they were teaching me things I didn't even know I was learning at the time.

#SoftwareEngineering #GameDevelopment #AI #Programming #LearningInPublic